JS-001 – Gyakorló feladatsor

Tematikus feladatsor – JavaScript

Rész	Feladat	Pont/feladat	Összesen
I. Nyelvi alapok és adatszerkezetek	25	2	50
II. DOM, események, aszinkron kód és modulok	25	2	50
Összesen	**50**		**100**

Tudnivalók

• A feladatsor megoldására 75 perc áll rendelkezésre.

• Minden feladatnál pontosan egy válasz helyes. Ha a választ kihagyja, vagy egynél több lehetőséget jelöl meg, az adott feladatra 0 pont jár.

• A megoldáshoz íróeszközön és nem programozható számológépen kívül más segédeszköz nem használható.

• A helyes válasz betűjelét karikázza be, vagy húzza alá. Javítani nem lehet.

I. Nyelvi alapok és adatszerkezetek — 50 pont

25 feladat, feladatonként 2 pont

1. Mit ír ki az alábbi kódrészlet? (2 pont)

```
console.log(x);
var x = 5;
```

a) undefined
b) ReferenceError hibát dob
c) null
d) 5

2. Mi történik az alábbi kód futtatásakor? (2 pont)

```
console.log(y);
let y = 5;
```

a) Kiírja az 5 értéket, mert a let is felkerül előre
b) Nem történik semmi, a sor figyelmen kívül marad
c) Kiírja az undefined értéket a konzolra
d) ReferenceError hibát dob, mert y még nem érhető el

3. Mi történik az alábbi kód futtatásakor? (2 pont)

```
const tomb = [1, 2];
tomb.push(3);
console.log(tomb);
```

a) Új tömböt hoz létre, az eredeti változatlan marad
b) Kiírja a [1, 2, 3] tömböt, mert a tartalom módosítható
c) Hibát dob, mert a const nem engedi a tömb módosítását
d) Kiírja az [1, 2] tömböt, a push hatástalan marad

4. Mit ad vissza az alábbi kifejezés? (2 pont)

```
typeof function () {}
```

a) "Function"
b) "undefined"
c) "function"
d) "object"

5. Mit ír ki az alábbi két sor? (2 pont)

```
console.log(null == undefined);
console.log(null === undefined);
```

a) true, majd true
b) false, majd true
c) true, majd false
d) false, majd false

6. Mit ad vissza az alábbi kifejezés? (2 pont)

```
Number("")
```

a) 0
b) undefined
c) NaN
d) null

7. Mit ad vissza az alábbi kifejezés? (2 pont)

```
parseInt("08")
```

a) 0
b) 8
c) "08" szövegként
d) NaN

8. Mit ír ki az alábbi sor, és miért? (2 pont)

```
console.log(0.1 + 0.2 === 0.3);
```

a) true, mert a motor a végeredményt automatikusan kerekíti
b) true, de csak szigorú módban fut le helyesen
c) false, mert a === nem használható számokra
d) false, mert a lebegőpontos ábrázolás pontatlan

9. Mit ad vissza az alábbi kifejezés? (2 pont)

```
"abcdef".slice(-2)
```

a) "cd"
b) "ef"
c) "ab"
d) Üres szöveget

10. Mit ad vissza az alábbi kifejezés? (2 pont)

```
[1, 2, 3].indexOf(4)
```

a) -1
b) false
c) 0
d) undefined

11. Mit eredményez az alábbi kódrészlet? (2 pont)

```
const t = [1, 2, 3, 4];
t.splice(1, 2);
console.log(t);
```

a) [1, 2, 3, 4] – a splice csak másolaton dolgozik
b) [2, 3] – a splice az eltávolított elemekkel tér vissza
c) [1, 4] – a splice módosítja az eredeti tömböt
d) [1, 3, 4] – a splice egyetlen elemet távolít el

12. Miben más a slice a splice metódustól? (2 pont)

a) A slice két tömböt fűz össze egyetlen tömbbé
b) A slice mindig az utolsó elemmel tér vissza
c) A slice csak számokat tartalmazó tömbre alkalmazható
d) A slice új tömböt ad vissza, az eredetit nem módosítja

13. Mit ad vissza az alábbi kifejezés? (2 pont)

```
Object.keys({ a: 1, b: 2 })
```

a) 2
b) [1, 2]
c) { a: 1, b: 2 }
d) ["a", "b"]

14. Mit eredményez az alábbi kódrészlet? (2 pont)

```
const o = { a: 1, b: 2 };
const u = { ...o, a: 9 };
console.log(u);
```

a) { a: 9 } – csak a megadott mező kerül bele
b) Hibát dob, mert az a mező kétszer szerepel
c) { a: 9, b: 2 } – másolat, felülírt a mezővel
d) { a: 1, b: 2 } – az eredeti érték marad érvényben

15. Mi a különbség a for...of és a for...in között? (2 pont)

a) A for...of aszinkron, a for...in szinkron módon fut
b) A for...of az értékeken, a for...in a kulcsokon lépked
c) A for...of csak tömbre, a for...in csak szövegre használható
d) A for...of visszafelé, a for...in előre halad a tömbön

16. Mit ad vissza az alábbi kifejezés? (2 pont)

```
[2, 4, 6].every(x => x % 2 === 0)
```

a) true, mert minden elem teljesíti a feltételt
b) Az első olyan elemet, amely páros
c) A feltételnek megfelelő elemek új tömbjét adja vissza
d) A páros elemek darabszámát a tömbben

17. Mit ad vissza az alábbi kifejezés? (2 pont)

```
[1, 3, 5].some(x => x > 4)
```

a) Az [5] tömböt a megfelelő elemekkel
b) false, mert nem minden elem nagyobb négynél
c) true, mert van legalább egy ilyen elem
d) Az 5 értéket, az első megfelelő elemet

18. Miért szükséges az összehasonlító függvény az alábbi hívásban? (2 pont)

```
[10, 9, 100].sort((a, b) => a - b);
```

a) Mert e nélkül az eredeti tömb nem módosulna
b) Mert alapértelmezésben szövegként rendezne
c) Mert a sort csak függvénnyel hívható meg
d) Mert a sort csak így ad vissza új tömböt

19. Mit ír ki az alábbi kódrészlet? (2 pont)

```
const { a = 5 } = {};
console.log(a);
```

a) Hibát dob, mert a mező nem létezik
b) undefined, mert a mező hiányzik az objektumból
c) null, mert az üres objektum nem ad értéket
d) 5 – az alapértelmezett érték lép életbe

20. Miben más a nyílfüggvény this kezelése a hagyományos függvényétől? (2 pont)

a) Nem kap saját this értéket, a külső környezetét örökli
b) A this értéke benne mindig undefined marad
c) Mindig a globális objektumra mutató this értéket kap meg
d) A this értékét a hívás helye szerint állítja be

21. Mikor fut le a finally ág az alábbi szerkezetben? (2 pont)

```
try {
  muvelet();
} catch (e) {
  naplo(e);
} finally {
  zaras();
}
```

a) Csak akkor, ha nem keletkezett hiba
b) Csak akkor, ha a catch ág lefutott
c) Minden esetben, hibától függetlenül
d) Csak akkor, ha a függvény értéket ad vissza

22. Mit eredményez az alábbi sor? (2 pont)

```
throw new Error("Hibás adat");
```

a) Kiírja a hibaüzenetet a konzolra, és a kód továbbfut
b) Visszatér a függvényből a megadott szöveggel
c) Leállítja a böngésző teljes működését azonnal
d) Kivételt dob, amelyet a hívó catch ága kaphat el

23. Mi történik az alábbi sor futtatásakor? (2 pont)

```
JSON.parse("{'a': 1}");
```

a) undefined értéket ad vissza csendben
b) Hibát dob, mert a JSON csak idézőjeles kulcsokat enged
c) Visszaadja az { a: 1 } objektumot hibátlanul, idézőjel nélkül is
d) Visszaadja a kapott szöveget változatlanul

24. Mit ír ki az alábbi kódrészlet? (2 pont)

```
const f = (a, b = 2) => a + b;
console.log(f(3));
```

a) 5
b) NaN
c) 3
d) Hibát dob, mert hiányzik a második érték

25. Mit ad vissza az alábbi kifejezés? (2 pont)

```
"5" * "2"
```

a) 10 számként
b) "52" szövegként
c) NaN
d) "10" szövegként

II. DOM, események, aszinkron kód és modulok — 50 pont

25 feladat, feladatonként 2 pont

1. Mit eredményez az alábbi kódrészlet? (2 pont)

```
const li = document.createElement("li");
li.textContent = "Új elem";
lista.appendChild(li);
```

a) Új listaelemet fűz a lista végére a megadott szöveggel
b) Új listát hoz létre, és abba teszi az elemet
c) Lecseréli a lista teljes tartalmát az új listaelemre
d) Az elemet a lista elejére szúrja be

2. Mit végez az alábbi sor? (2 pont)

```
gomb.setAttribute("disabled", "");
```

a) Törli a gomb összes korábbi attribútumát
b) Letiltja a gombot az attribútum beállításával
c) Eltávolítja a gombot a dokumentum szerkezetéből
d) Elrejti a gombot, de kattintható marad

3. Mit ad vissza az alábbi kifejezés? (2 pont)

```
<li data-azonosito="42">Tétel</li>

elem.dataset.azonosito
```

a) undefined
b) 42 számként
c) "42" szövegként
d) A teljes data attribútum objektumot

4. Mit eredményez az alábbi kódrészlet? (2 pont)

```
urlap.addEventListener("submit", e => {
  e.preventDefault();
  kuldes();
});
```

a) Az űrlap elküldése után fut le a saját kód
b) Az űrlap csak egyszer küldhető el a lapon
c) Az esemény nem jut el a szülő elemekhez a dokumentumban
d) Elmarad az űrlap alapértelmezett elküldése, helyette a saját kód fut

5. Mit akadályoz meg az e.stopPropagation() hívás? (2 pont)

a) Hogy az esemény bekerüljön a naplóba
b) Hogy az elem alapértelmezett viselkedése lefusson
c) Hogy az esemény továbbterjedjen a szülőelemek felé
d) Hogy az eseménykezelő többször is lefusson

6. Mi a különbség az e.target és az e.currentTarget között? (2 pont)

a) A target az eseményt kiváltó elem, a currentTarget a kezelő gazdája
b) A target a szülő, a currentTarget mindig a dokumentum
c) A target az új, a currentTarget a korábbi állapotot adja
d) A target csak kattintásnál, a currentTarget mindig elérhető érték

7. Melyik eseményre érdemes figyelni, ha a mező tartalmát gépelés közben követnénk? (2 pont)

a) "change"
b) "blur"
c) "submit"
d) "input"

8. Mit eredményez az alábbi kódrészlet? (2 pont)

```
const id = setInterval(lep, 1000);
// ...
clearInterval(id);
```

a) A függvényt a lap bezárásáig biztosan futtatja
b) Másodpercenként hívja a függvényt, amíg le nem állítjuk
c) Egyszer, egy másodperc múlva hívja meg a függvényt
d) Ezredmásodpercenként hívja meg a függvényt ezerszer

9. Mikor teljesül az alábbi kifejezés eredménye? (2 pont)

```
const eredmeny = await Promise.all([ keres1, keres2 ]);
```

a) Ha a leggyorsabb ígéret megérkezett
b) Ha legalább az egyik ígéret teljesült már a kettő közül
c) Ha mindkét ígéret elutasításra került
d) Ha mindkét ígéret teljesült, vagy az első hibánál elutasít

10. Melyik ág fut le hiba esetén az alábbi láncban? (2 pont)

```
fetch(url)
  .then(v => v.json())
  .then(adat => megjelenit(adat))
  .catch(hiba => naplo(hiba));
```

a) A második then ág, hibajelzéssel a paraméterben
b) Egyik sem, a hiba csendben elvész a láncban
c) A catch ág, amely a lánc bármely hibáját elkapja
d) A catch ág, de csak a fetch hibáit kapja el

11. Hogyan kezeljük a hibát async függvényben? (2 pont)

```
async function betolt() {
  try {
    const v = await fetch(url);
  } catch (e) {
    naplo(e);
  }
}
```

a) Az await elé tett kérdőjeles operátorral
b) try-catch szerkezettel, akár a szinkron kódban
c) Kizárólag .catch() hívással a függvény után
d) A függvény elé írt onerror kezelővel

12. Miért kell ellenőrizni a válasz ok tulajdonságát az alábbi kódban? (2 pont)

```
const valasz = await fetch(url);
if (!valasz.ok) { /* ... */ }
```

a) Mert a fetch a HTTP hibakódoknál nem dob kivételt
b) Mert a fetch mindig sikeresnek jelzi a hálózati hibát
c) Mert az ok mező nélkül a json() nem hívható meg
d) Mert a válasz törzse csak így olvasható ki

13. Mit küld el az alábbi kérés? (2 pont)

```
await fetch("/api/diak", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ nev: "Kiss Anna" })
});
```

a) Űrlapként kódolt POST kérést küld a megadott végpontra
b) GET kérést, a paramétereket az URL-ben továbbítva
c) JSON törzzsel ellátott POST kérést a megadott végpontra
d) PUT kérést, amely a meglévő diákot módosítja

14. Milyen típusú érték kerül a tárolóba az alábbi sorban? (2 pont)

```
localStorage.setItem("beallitas", { sotet: true });
```

a) Logikai érték, az objektum igazságértéke
b) Objektum, a mezőivel és azok értékeivel együtt megőrizve
c) Semmi, mert objektum nem tárolható így
d) Szöveg, mégpedig az objektum [object Object] alakja

15. Mit eredményez az alábbi sor? (2 pont)

```
const b = JSON.parse(localStorage.getItem("beallitas"));
```

a) A tárolt szövegből objektumot állít elő
b) A tárolt objektumot szöveggé alakítja
c) Új kulcsot hoz létre a tárolóban
d) Törli a tárolóból a megadott kulcsot

16. Hogyan importáljuk az alábbi modul alapértelmezett exportját? (2 pont)

```
// szamolo.js
export default function osszead(a, b) { return a + b; }
```

a) require("./szamolo.js").osszead;
b) import osszead from "./szamolo.js";
c) import * as osszead from "./szamolo.js";
d) import { osszead } from "./szamolo.js";

17. Mit jelent az alábbi import forma? (2 pont)

```
import { formaz, kerekit } from "./util.js";
```

a) A modul alapértelmezett exportját nevezi át
b) Két külön modult tölt be egyetlen sorban
c) A modul két nevesített exportját hozza be
d) A modul teljes tartalmát két néven hozza be

18. Mi jellemző az alábbi módon betöltött szkriptre? (2 pont)

```
<script type="module" src="fo.js"></script>
```

a) Azonnal lefut, a HTML feldolgozását közben megszakítva
b) Modulként tölti be, és a HTML feldolgozása után fut le
c) Minden más szkript előtt, elsőként fut le
d) Csak akkor fut le, ha a felhasználó rákattint

19. Mit eredményez az alábbi kódrészlet? (2 pont)

```
class Auto {
  constructor(marka) { this.marka = marka; }
}
const a = new Auto("Suzuki");
```

a) Hibát dob, mert az osztálynak nincs metódusa
b) Létrehoz egy osztályt, de példányt nem készít belőle
c) Globális változót hoz létre marka néven
d) Létrehoz egy példányt, amelynek marka mezője Suzuki

20. Mire való a super() hívás az alábbi konstruktorban? (2 pont)

```
class Teherauto extends Auto {
  constructor(marka, teher) {
    super(marka);
    this.teher = teher;
  }
}
```

a) Meghívja az ősosztály konstruktorát a megadott adattal
b) Létrehoz egy másik példányt az ősosztályból a mezőkkel
c) Megakadályozza az ősosztály példányosítását
d) Felülírja az ősosztály összes metódusát

21. Hogyan érhető el az alábbi osztály teruletben megadott érték? (2 pont)

```
class Telek {
  constructor(a, b) { this.a = a; this.b = b; }
  get terulet() { return this.a * this.b; }
}
```

a) telek.terulet() alakban, metódusként hívva
b) Csak az osztályon belülről érhető el
c) telek.get('terulet') hívással
d) telek.terulet alakban, zárójel nélkül

22. Mi történik az alábbi kódrészlet utolsó sorában? (2 pont)

```
class Fiok {
  #egyenleg = 0;
}
const f = new Fiok();
console.log(f.#egyenleg);
```

a) Hibát dob, mert a privát mező kívülről nem érhető el
b) Kiírja a 0 értéket, mert a mező kívülről is olvasható
c) undefined értéket ír ki a konzolra
d) Létrehozza a mezőt, és nulla értéket ad neki

23. Miért működik az alábbi megoldás később hozzáadott elemekre is? (2 pont)

```
lista.addEventListener("click", e => {
  const gomb = e.target.closest("button");
  if (gomb) torol(gomb);
});
```

a) Mert a closest minden új elemre újra lefut betöltéskor
b) Mert a kezelő a szülőn figyel, és onnan azonosítja a forrást
c) Mert a click esemény nem terjed tovább a szülő felé
d) Mert a böngésző automatikusan átköti az eseménykezelőket

24. Mit eredményez az alábbi másolás? (2 pont)

```
const a = { nev: "A", cim: { varos: "Pécs" } };
const b = { ...a };
b.cim.varos = "Szeged";
console.log(a.cim.varos);
```

a) "Pécs", mert a szórásoperátor mélymásolatot készít
b) undefined, mert a cim mező nem másolódott át
c) "Szeged", mert a beágyazott objektum közös marad
d) Hibát dob, mert az objektum nem módosítható

25. Mit jelent, hogy a JavaScript egyszálú, eseményvezérelt nyelv? (2 pont)

a) Minden függvény külön folyamatban fut a böngészőben
b) A kód csak felhasználói esemény hatására indul el a lapon
c) Egyszerre egy kódrészlet fut, a kész feladatok sorban kerülnek rá
d) Egyszerre több szálon futtatja a függvényeket